

LC 1568: Minimum Number of Days to Disconnect Island

Authored by Arpit Raj, LNMIIT Jaipur

© July 22, 2026. All rights reserved.

Algorithmic Approach

Step 1: Initial Assessment

Count the total number of islands currently present in the grid.

- If the count is `0` or `≥ 2`, the grid is already disconnected. Return `0`.

Step 2: Try 1-Day Removal

If there is exactly `1` island, simulate removing 1 land cell at a time.

- If the number of islands after removing a single cell becomes `≠ 1`, then that single removal successfully disconnected the grid. Return `1`.

Step 3: Guaranteed Disconnection

If the grid cannot be disconnected by removing 0 or 1 cell, the answer is guaranteed to be `2` and will never exceed 2.

Graph Theory & Complexity Analysis

Time Complexity

$$TC = O((N \times M)^2)$$

Reasoning: After removing each land cell sequentially, we count the number of islands from scratch using Depth First Search (DFS).

Grid Graph Properties

- Let every land cell = **Vertex**
- Let adjacent connections = **Edge**

Our graph is a grid graph, where the vertex connectivity is strictly ≤ 2 . Because of the maximum degree of any node in a 2D grid, there always exists one or two cells whose removal fully disconnects the island.

Implementation (C++)

```
class Solution {
public:
    vector<pair<int,int>> dir = {{-1,0}, {0,-1}, {1,0}, {0,1}};
    int n;
    int m;

    void dfs(int r, int c, vector<vector<int>>& grid) {
        grid[r][c] = 0;
        for(auto it : dir) {
            int nr = r + it.first;
            int nc = c + it.second;
            if(nr >= 0 && nc >= 0 && nr < n && nc < m) {
                if(grid[nr][nc] == 1) {
                    grid[nr][nc] = 0;
                    dfs(nr, nc, grid);
                }
            }
        }
    }

    int countislands(vector<vector<int>> grid) { // pass by value we dont want to change orig
        int cnt = 0;
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(grid[i][j] == 1) {
                    cnt++;
                    dfs(i, j, grid); // to mark all adjacent as visited
                }
            }
        }
        return cnt;
    }

    int minDays(vector<vector<int>>& grid) {
        n = grid.size();
        m = grid[0].size();

        int cnt = countislands(grid);
        if(cnt == 0 || cnt >= 2) {
            return 0; // as already disconnected or nothing to solve
        }

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(grid[i][j] == 1) {
                    grid[i][j] = 0;
                    cnt = countislands(grid);
                    if(cnt >= 2 || cnt == 0) return 1;
                }
            }
        }
    }
};
```

```
        grid[i][j] = 1; // restore original for giving chance to next
        land -> water convert
    }
}

return 2; // agar 1 din me kaam nhi hua to 2 din me guarenteed ho jaayega
// according to graph theory for a grid graph , max 2 changes are
required to disconnect them
// reason degree <= 4 and vertex connexctivity <= 2;
}
};
```